Project Handoff Document

Screener Bot

June 7, 2026

A production-oriented Python monitoring bot that scrapes Screener.in latest quarterly results, calculates alert conditions for Sales, PAT Margin %, EBITDA Margin %, and latest-results YOY changes, persists state, and sends consolidated Telegram alerts.

1. PROJECT OVERVIEW

Project name: Screener Bot
Purpose: Monitor Screener.in latest quarterly result updates, filter companies by market cap, extract key financial metrics, calculate threshold-based changes, persist state to avoid duplicate alerts, and notify static and dynamic Telegram subscribers.
Tech stack: Python 3, asyncio, Playwright, BeautifulSoup, urllib, python-dotenv, Azure Blob Storage SDK, Docker, GitHub Actions, Telegram Bot API.

Current status: The codebase is compact and operational. It has clear module boundaries for scraping, parsing, alert formatting, storage, and Telegram notification. It includes Docker packaging and GHCR image publishing. Recent logic supports Azure Blob backed JSON state, Telegram getUpdates polling, dynamic subscribers, market-cap filtering, partial quarterly metric handling, and YOY-based alerting.

2. ARCHITECTURE DIAGRAM

```
Azure Container App / local Python process
|
+-- main.py
|   +-- loads .env values in local mode
|   +-- polls Telegram getUpdates before/after scans and during sleep
|   +-- runs scan cycle every SCAN_INTERVAL_SECONDS
|
+-- scraper.py (Playwright)
|   +-- opens https://www.screener.in/results/latest/
|   +-- logs in with Screener credentials when required
|   +-- scrolls latest-results cards
|   +-- opens each company #quarters page
|
+-- parser.py (BeautifulSoup + calculations)
|   +-- filters M.Cap >= 200 Cr
|   +-- extracts Sales, Operating Profit/EBITDA/EBIDT, Net Profit/PAT
|   +-- computes Sales, PAT Margin %, EBITDA Margin %, YOY margin changes
|
+-- alerter.py
|   +-- formats one consolidated company alert
|
+-- telegram_notifier.py
|   +-- static recipients from TELEGRAM_CHAT_ID / TELEGRAM_CHAT_IDS
|   +-- dynamic subscribers via /start and /stop
|   +-- sends messages through Telegram Bot API
|
+-- storage.py
    +-- local JSON files or Azure Blob JSON blobs
    +-- state.json, telegram_subscribers.json, telegram_offset.json
```

Data flow: Screener latest results -> company links and YOY values -> company quarterly pages -> parsed financial metrics -> alert calculations -> state comparison -> console output and Telegram alert -> state saved.

3. FOLDER STRUCTURE

```
screener_bot/
|-- .github/
|   `-- workflows/docker-publish.yml   CI workflow to build and push Docker image to GHCR.
|-- docs/
|   |-- goal.md                        Functional scrape/alert behavior specification.
|   `-- bot.json                       Unrelated/legacy n8n workflow document.
|-- alerter.py                         Alert text formatting and console printing.
|-- main.py                            Async entry point, scan loop, CLI flags, state orchestration.
|-- parser.py                          HTML parsing, market-cap filtering, calculations.
|-- scraper.py                         Playwright browser setup, login, listing scrape, company scrape.
|-- storage.py                         Local JSON and Azure Blob JSON persistence abstraction.
|-- telegram_notifier.py               Telegram send, getUpdates polling, dynamic subscribers.
|-- requirements.txt                   Python runtime dependencies.
|-- Dockerfile                         Production container definition.
|-- .env.example                       Placeholder local configuration template.
`-- .dockerignore                      Excludes local secrets/state from Docker build context.
```

4. SETUP & INSTALLATION

Prerequisites: Python 3.12+ recommended, pip, Playwright browser dependencies, Git, Docker for container builds, and a Telegram bot token. Azure deployment requires Azure Container Apps and Azure Blob Storage.

```
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
python -m playwright install chromium
copy .env.example .env
# Edit .env with Screener credentials and Telegram settings.
python main.py --once --limit 1 --headless
```

Development mode: Run `python main.py --once --limit 1 --headed` for a visible browser test, or run `python main.py --headless` for continuous monitoring. Use `--company-url` and `--company-name` for direct one-company tests.

5. ENVIRONMENT VARIABLES

Variable Name	Purpose	Example Value	Required
TELEGRAM_BOT_TOKEN	Telegram Bot API token used for sendMessage and getUpdates.	replace_with_bot_token	Y for Telegram
TELEGRAM_CHAT_ID	Single static Telegram recipient.	replace_with_chat_id	N
TELEGRAM_CHAT_IDS	Comma-separated static seed recipients.	optional_comma_separated_seed_chat_ids	N
TELEGRAM_SUBSCRIBERS_FILE	Local fallback subscriber JSON file name.	telegram_subscribers.json	N
SUBSCRIBERS_BLOB_NAME	Blob/local name for dynamic subscriber JSON.	telegram_subscribers.json	N
TELEGRAM_OFFSET_BLOB_NAME	Blob/local name for Telegram getUpdates offset JSON.	telegram_offset.json	N
TELEGRAM_TIMEOUT_SECONDS	HTTP timeout for Telegram calls.	10 or 15	N
SCREENER_USERNAME	Screener login username/email.	replace_with_screener_email_or_username	Y if login required
SCREENER_EMAIL	Alternative Screener login env var accepted by scraper.	user@example.com	N
SCREENER_PASSWORD	Screener login password.	replace_with_screener_password	Y if login required
SCAN_INTERVAL_SECONDS	Seconds between scan cycles.	300	N
STATE_BACKEND	Persistence backend selector: blob uses Azure Blob; otherwise local JSON.	blob	N
STATE_BLOB_NAME	Name/path for state JSON.	state.json	N
AZURE_STORAGE_CONNECTION_STRING	Azure Storage connection string for blob backend.	configured as Azure secret	Y when STATE_BACKEND=blob
AZURE_STORAGE_CONTAINER	Azure Blob container name.	screener-bot-state	Y when STATE_BACKEND=blob
HEADLESS	Expected deployment setting for browser mode; CLI currently controls runtime headless flag.	true	Deployment convention
MAX_CONCURRENT_COMPANY_PAGES	Expected Azure setting; current code processes companies sequentially.	1	N

6. API / INTERFACE REFERENCE

Module / Interface	Purpose	Inputs / Outputs
main.py CLI	Runs the bot, test modes, and direct company checks.	--headless/--headed, --once, --limit, --company-url, --company-name, --telegram-test, --telegram-chat-id.
main.run_cycle()	Processes one list of companies, evaluates alerts, sends notifications, and saves state.	Inputs: Playwright page, cycle number, state dict. Output: bool alerts_detected.
scraper.scrape_latest_companies()	Loads Screener latest results, logs in if needed, scrolls cards, returns filtered company records.	Returns list of dicts with id, name, url, market_cap_cr, yoy.
scraper.scrape_company_metrics_with_retries()	Fetches a company #quarters page with retry handling.	Returns dict of MetricQuarterValues keyed by sales/op_profit/net_profit when available.
parser.parse_latest_results_companies()	Parses listing page HTML, filters non-company PDF links and M.Cap under 200 Cr.	Input HTML; output company records and latest-results YOY metrics.
parser.build_alert_metrics()	Calculates Sales change, PAT Margin % change, EBITDA Margin % change when source values exist.	Output dict of AlertMetricValues with triggered flags.
parser.yoy_alert_needed()	Checks if Sales YOY, EBITDA margin YOY, or PAT margin YOY crosses abs(20%).	Input YOY metric dict; output bool.
alerter.format_combined_alert()	Builds one consolidated human-readable alert per company.	Output Telegram-safe plain text alert.
telegram_notifier.poll_telegram_subscribers_once()	Polls Telegram getUpdates, handles /start and /stop, saves offset.	Returns processed update count.
telegram_notifier.send_telegram_alert_to_all()	Sends one alert to unique static and dynamic recipients.	Input message string; output bool success across all recipients.
storage.get_storage()	Selects LocalJsonStorage or AzureBlobJsonStorage based on STATE_BACKEND.	Provides load_json and save_json.

7. THIRD-PARTY SERVICES & INTEGRATIONS

Service	Usage	Authentication / Configuration
Screener.in	Source website for latest quarterly results and company quarterly tables.	SCREENER_USERNAME or SCREENER_EMAIL plus SCREENER_PASSWORD when login is required.
Telegram Bot API	Dynamic subscriber polling through getUpdates and alert delivery through sendMessage.	TELEGRAM_BOT_TOKEN plus static chat IDs or dynamic subscribers.
Azure Blob Storage	Persistent JSON state for company state, Telegram subscribers, and update offsets in cloud deployments.	AZURE_STORAGE_CONNECTION_STRING and AZURE_STORAGE_CONTAINER.
Azure Container Apps	Target runtime for the Dockerized worker with ingress disabled.	Configured externally through Azure Portal secrets/env vars.
GitHub Container Registry	Stores the built Docker image.	GitHub Actions uses GITHUB_TOKEN in docker-publish.yml.
Playwright Chromium	Browser automation for Screener navigation and scraping.	Docker image includes browser runtime; local install uses playwright install chromium.

8. DEPLOYMENT GUIDE

Docker production build:

```
docker build -t ghcr.io/name-nikhilpatil/screener-bot:latest .
docker push ghcr.io/name-nikhilpatil/screener-bot:latest
```

CI/CD: The GitHub Actions workflow at `.github/workflows/docker-publish.yml` builds and pushes the Docker image to GHCR on pushes to `main` and manual workflow dispatch.

Azure Container Apps deployment shape:

Run the image as a worker-style Container App with ingress disabled. Configure secrets/env vars in Azure Portal, including Telegram token, Screener credentials, Azure Storage connection string, container name, state blob names, and scan interval. Use Azure Blob Storage for state persistence by setting `STATE_BACKEND=blob`.

Recommended Azure runtime variables:

```
STATE_BACKEND=blob
AZURE_STORAGE_CONTAINER=screener-bot-state
STATE_BLOB_NAME=state.json
SUBSCRIBERS_BLOB_NAME=telegram_subscribers.json
TELEGRAM_OFFSET_BLOB_NAME=telegram_offset.json
SCAN_INTERVAL_SECONDS=300
HEADLESS=true
```

Initial blob contents: Create `telegram_subscribers.json` with `[]` and `telegram_offset.json` with `{"offset": 0}`. The storage layer also creates missing blobs using defaults when possible.

Operational smoke test: after a new revision starts, send `/start` to the Telegram bot, watch logs for getUpdates processing, verify the subscriber blob changed, then allow the next scan cycle to run.